III B.Tech I Semester

FORMAL LANGUAGES AND AUTOMATA THEORY (R23)

UNIT-V

Objective

To learn how to design Turing Machines

Syllabus

Module 1:

Turing Machine: Definition ,Model, Representation of TMs, Instantaneous Descriptions, Transition Tables and Transition Diagrams, Language of a TM, Design of TMs, Types of TMs, Church's Thesis, Universal and Restricted TM.

Module 2:

Decidable and Un-decidable Problems, Halting Problem of TMs, Post's Correspondence Problem, Modified PCP, Classes of P and NP, NP-Hard and NP Complete Problems.

Learning Outcomes:

Students will able to

- Understand Turing Machine Fundamentals
- Design Turing Machines
- Analyze Undecidable Problems
- Classify Classes of P and NP, NP-Hard and NP Complete Problems

Module 1:

5.1 Turing Machine

5.1.1 Definition

A Turing machine (TM) is denoted by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F),$$

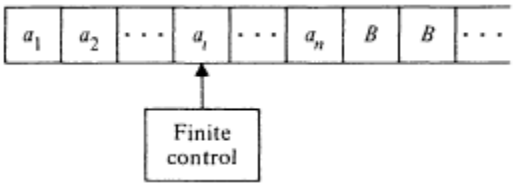
where

Q is the finite set of *states*,
Γ is the finite set of allowable *tape symbols*,
B, a symbol of Γ, is the *blank*,
Σ, a subset of Γ not including B, is the set of *input symbols*,
δ is the *next move function*, a mapping from $Q \times \Gamma$ to $Q \times \Gamma \times \{L, R\}$ (δ may, however, be undefined for some arguments),
q_0 in Q is the *start state*,
$F \subseteq Q$ is the set of *final states*.

5.1.2 Turing Machine Model

- The basic model has a finite control, an input tape that is divided into cells, and a tape head that scans one cell of the tape at a time.
- The tape has a leftmost cell but is infinite to the right. Each cell of the tape may hold exactly one of a finite number of tape symbols.

- Initially, the n leftmost cells, for some finite n ≥ 0, hold the input, which is a string of symbols chosen from a subset of the tape symbols called the input symbols.
- The remaining infinity of cells each hold the blank, which is a special tape symbol that is not an input symbol.



Moves of Turning Machine

In one move the Turing machine, depending upon the symbol scanned by the tape head and the state of the finite control,

1) changes state,
2) prints a symbol on the tape cell scanned, replacing what was written there, and
3) moves its head left or right one cell.

Note : The difference between a Turing machine and a two-way finite automaton lies in the former's ability to change symbols on its tape.

5.2 Representation of Turing Machine

5.2.1. Instantaneous description (ID):

- Instantaneous description of the Turing machine M is denoted by α1qα2.
- Here q, the current state of M, is in Q; α1α2 is the string in Γ* that is the contents of the tape up to the rightmost nonblank symbol or the symbol to the left of the head, whichever is rightmost. (Observe that the blank B may occur in α1α2.).
- The tape head is assumed to be scanning the leftmost symbol of α2, or if α2 = ε, the head is scanning a blank.

5.2.2. Transition Tables and Transition Diagrams

- A Transition Table and Transition Diagram are two methods used to visualize the transition function of a Turing machine.
- They define the machine's behavior: given a current state and a current symbol read on the tape, the machine will transition to a new state, write a new symbol, and move its head.

Transition Table

A Transition Table is a tabular representation of the transition function of a Turing Machine (TM). It specifies the action the TM should take for every combination of current state and current tape symbol.

The transition function is represented as: **δ(q, X)=(p, Y, D)**

where:

- **q** – Current state
- **X** – Symbol currently read

- **p** – Next state
- **Y** – Symbol written on the tape
- **D** – Direction of head movement (L = Left, R = Right)

Transition Diagrams

A Transition Diagram is a graphical representation of a Turing Machine in which:

- States are represented by circles.
- Transitions are represented by directed arrows.
- Each arrow is labeled with the format: Read / Write, Move

5.3 Language of a Turing Machine

- The language accepted by M, denoted L(M), is the set of those words in ∑* that cause M to enter a final state when placed, justified at the left, on the tape of M, , with M in state q0, and the tape head of M at the leftmost cell.
- Formally, the language accepted by M = (Q,∑,Γ,δ,q0,B,F) is

$$\{w \mid w \text{ in } \Sigma^* \text{ and } q_0 w \vdash^* \alpha_1 p \alpha_2 \text{ for some } p \text{ in } F, \text{ and } \alpha_1 \text{ and } \alpha_2 \text{ in } \Gamma^*\}.$$

5.4 Design of Turing Machines

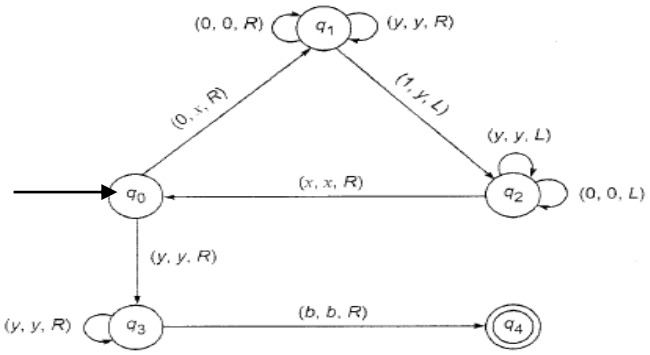
Example:

Design a TM to accept the language L = {0^n1^n | n≥1}.

- Initially, the tape of M contains 0^n1^n followed by infinity of blanks.
- Repeatedly, M replaces the leftmost 0 by X, moves right to the leftmost 1, replacing it by Y,, moves left to find the rightmost X, then moves one cell right to the leftmost 0 and repeats the cycle.
- If, however, when searching for a 1, M finds a blank instead, , then M halts without accepting.
- If, after changing a 1 to a Y, M finds no more 0's, then M checks that no more 1's remain, accepting if there are none.

State	Symbol 0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

The function δ



Transition Diagram

String Verfication by Turning Machine

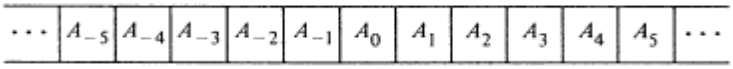
$$
\begin{aligned}
q_0 0011 &\vdash X q_1 011 \vdash X 0 q_1 11 \vdash X q_2 0 Y 1 \vdash \\
q_2 X 0 Y 1 &\vdash X q_0 0 Y 1 \vdash X X q_1 Y 1 \vdash X X Y q_1 1 \vdash \\
X X q_2 Y Y &\vdash X q_2 X Y Y \vdash X X q_0 Y Y \vdash X X Y q_3 Y \vdash \\
X X Y Y q_3 &\vdash X X Y Y B q_4
\end{aligned}
$$

A computation of M

5.5 Types of Turing Machines

1. Two-way infinite tape:

A Turing machine with a two-way infinite tape is denoted by M = (Q,∑,Γ,δ,q0,B,F). As its name implies, the tape is infinite to the left as well as to the right. We denote an ID of such a device as for the one-way infinite TM. We imagine, however, that there is an infinity of blank cells both to the left and right of the current nonblank portion of the tape.



2. Multitape Turing machines:

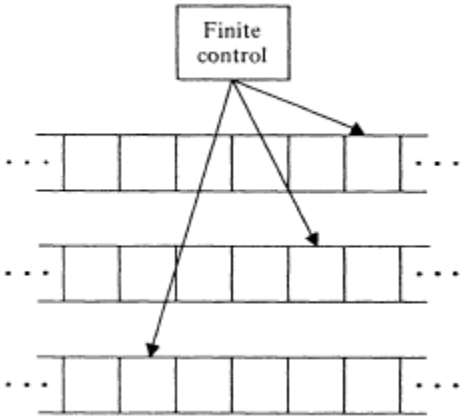
A multitape Turing machine consists of a finite control with k tape heads and k tapes; each tape is infinite in both directions.. On a single move, depending on the state of the finite control and the symbol scanned by each of the tape heads, the machine can:

1) change state;

2) print a new symbol on each of the cells scanned by its tape heads;

3) move each of its tape heads,, independently, one cell to the left or right, or keep it stationary.

Initially, the input appears on the first tape, and the other tapes are blank.



3. Nondeterministic Turing machines:

A nondeterministic Turing machine is a device with a finite control and a single, one-way infinite tape. For a given state and tape symbol scanned by the tape head, the machine has a finite number of choices for the next move. Each choice consists of a new state, a tape symbol to print, and a direction of head motion. Note that the nondeterministic TM is not permitted to make a move in which the next state is selected from one choice, and the symbol printed and/or direction of head motion are selected from other choices. The nondeterministic TM accepts its input if any sequence of choices of moves leads to an accepting state.

4. Multidimensional Turing machines:

The device has the usual finite control, but the tape consists of a k-dimensional array of cells infinite in all 2k directions, for some fixed k. Depending on the state and symbol scanned, the device changes state, prints a new symbol, and moves its tape head in one of 2k directions, either positively or negatively, along one of the k axes. Initially, the input is along one axis, and the head is at the left end of the input. At any time, only a finite number of rows in any dimension contain nonblank symbols, and these rows each have only a finite number of nonblank symbols.

B	B	B	a_1	B	B	B
B	B	a_2	a_3	a_4	a_5	B
a_6	a_7	a_8	a_9	B	a_{10}	B
B	a_{11}	a_{12}	a_{13}	B	a_{14}	a_{15}
B	B	a_{16}	a_{17}	B	B	B

5. Multihead Turing machines:

A k-head Turing machine has some fixed number,,k, of heads. The heads are numbered 1 through k,, and a move of the TM depends on the state and on the symbol scanned by each head.. In one move, the heads may each move independently left,, right, or remain stationary.

6. Off-line Turing machines:

An off-line Turing machine is a multitape TM whose input tape is read-only. Usually we surround the input by endmarkers, ⊄ on the left and $ on the right. The Turing machine is not allowed to move the input tape head off the region between ⊄ and $.

5.6 Church's Thesis

The Church-Turing Thesis is an important idea in the study of computability i.e. the ability to solve problems using a set of rules or procedures. It is an abstract model of a computing device, proposed by **Alan Turing and Alonzo Church**. It helps define algorithms and computing processes.

The basic idea of the thesis is:

- Any effective calculation or computation that can be done by a human (following specific steps) can also be done by a Turing machine.
- A Turing machine is a simple mathematical model that represents a basic form of a computer. This machine helps explain the logic behind modern computers.

To define these algorithms clearly, Alonzo Church developed a method called "M" for manipulating strings using logic and mathematics. This method must meet the following criteria:

1. **Finite instructions**: The method must have a limited number of steps.
2. **Finite output**: The method should produce a result after a certain number of steps.
3. **Real-life feasibility**: The method should be physically possible.
4. **Simple to understand**: It should not require complex understanding.

Based on these conditions, Church proposed the Church-Turing Thesis, which states:

"Every computation that can be done in the real world can be effectively performed by a Turing machine."

- This idea was first formulated by Church in 1930 and is known as the Church-Turing Thesis. Although it cannot be proven, the hypothesis assumes that all computable functions can be represented by partial recursive functions.

In simpler terms:

- **Assumption 1**: Every function must be computable.
- **Assumption 2**: If a function **F** is computable and you perform basic operations on it to get a new function **G**, then **G** is also computable.

5.7 Universal and Restricted TM

Universal Turing Machine (UTM)

Purpose: To simulate any other Turing machine.

Functionality: It takes the description of another Turing machine (M) and its input (X) and then executes M on X, providing the output.

Structure: A UTM typically has three tapes: one for the description of the simulated machine, one for the input/data, and one to track the state of the simulated machine.

Significance: It is the theoretical basis for modern computers, demonstrating that a single machine can perform any task that can be described by an algorithm.

Restricted Turing Machine

Purpose: To study a limited set of computational problems.

Functionality: It has constraints on its operation compared to a standard or universal TM. Examples include:

- **Halting Turing Machine**: Always halts for every input, but can only accept a subset of languages.
- **Unidirectional Turing Machine**: The head can only move in one direction.
- **Linear Bounded Automata**: The tape storage is limited to the input's length.

Significance: By restricting the capabilities of the model, we can understand the boundaries of computation and design simpler, more specialized machines.

Module 2:

5.8 Decidable and Un-decidable Problems

- A problem whose language is recursive is said to be decidable.
- A problem is undecidable if there is no algorithm that takes as input an instance of the problem and determines whether the answer to that instance is "yes" or "no."

5.9 Halting Problem of TMs

The Halting Problem for Turing Machines (TMs) is the undecidable problem of determining whether an arbitrary Turing machine will eventually halt (stop) or run forever on a given input. Alan Turing proved this problem is undecidable, meaning no general algorithm or Turing machine can solve it for all possible Turing machine-input pairs.

5.10 Post's Correspondence Problem

An instance of Post's Correspondence Problem (PCP) consists of two lists, A = w1,, ..., wk and B = x1, ... , xk, of strings over some alphabet ∑. This instance of PCP has a solution if there is any sequence of integers i1, i2, .. ., im, with m ≥1, such that ,wi1, wi2,…, wim = xi1, xi2,… xim.

The sequence i1,…,im is a solution to this instance of PCP.

Example 1:

Let ∑ = {0, 1}. Let A and B be lists of three strings each, as defined

	List *A*	List *B*
i	w_i	x_i
1	1	111
2	10111	10
3	10	0

In this case PCP has a solution. Let m = 4, i1 = 2,i2 =1, i3 = 1, and i4 = 3. Then W2W1W1W3 = X2X1X1X3 = 101111110.

Example 2: Show that PCP problem with 2 lists

X=(b,bab^3,ba) and y=(b^3,ba,a) has a solution.

Given lists are x=(b,bab^3,ba) y=(b^3,ba,a)

The instances of PCP is as follows

	List X	List Y
i	X_i	Y_i
1	a	b^3
2	bab^3	ba
3	ba	a

In this case PCP is as follows

X2x1x1x3=y2y1y1y3=bab^{3}bbba

The solution sequence is 2113 PCP has a solution.

Example 3: Prove that PCP with two lists X =(01,1,1) Y=(0101,10,11) has no solution.

sol) Instance of PCP is given as

	List X	List Y
i	X_i	Y_i
1	01	0101
2	1	10
3	1	11

Where X1=01 Y1=0101

X2=1 Y2=10

X3=1 Y3=11

For any i $|X_i| < |Y_i|$The last Y is having strings of greater lengths. So to get same string for same sequences of x_1,x_2,x_3 and y_1,y_2,y_3 is difficult.

We cannot get solution sequence. Therefore the given PCP is having no solution.

Modified PCP

Modified Post Correspondence Problem (MPCP) is a variation of the Post Correspondence Problem (PCP) where the first pair of strings in the solution must be the first pair from the provided lists. While the standard PCP involves finding a sequence of indices to make the concatenated strings from two lists equal, MPCP adds the constraint that the first chosen pair from list A and list B must begin the matching sequence. This modification is often used as an intermediate step when reducing the Universal Language Problem to the PCP.

5.11 Classes of P and NP, NP-Hard and NP Complete Problems

Polynomial Time (P)

Polynomial time means that the algorithm's running time grows **at a manageable rate** as the input size increases. In other words, an algorithm is **efficient** if it can solve the problem in **polynomial time**.

For example:

- An algorithm with time complexity of $\mathbf{O(n^2)}$, where **n** is the input size, is still considered polynomial time because $\mathbf{n^2}$ is a polynomial function of **n**.

NP

- **NP** stands for **Nondeterministic Polynomial time**. This is a class of problems that can be **verified** in **polynomial time**. That means, if you're given a **candidate solution** to a problem, you can check if it's correct quickly (in a reasonable amount of time, using a polynomial function of the input size).
- However, just because you can **verify** a solution quickly, doesn't mean you can **find** the solution quickly. For some problems, finding the solution might take much longer.

Example:

- **Sudoku**: Given a completed Sudoku puzzle, you can verify if it's solved correctly in a reasonable amount of time. But finding the solution, when you have to fill in the numbers, is much more difficult (it may take much longer).

NP-complete

- **NP-complete** problems are a special subset of NP problems. A problem is NP-complete if:
    1. **It is in NP**: This means you can **verify** a solution quickly.
    2. **Every other NP problem can be reduced to it**: This means that, if you could solve this NP-complete problem quickly, you could solve all other NP problems quickly too.
- In simpler terms: NP-complete problems are the "hardest" problems in NP. If you can solve one NP-complete problem quickly, it would mean you can solve all NP problems quickly (which is a big deal!).

Example:

- **The Traveling Salesman Problem (TSP)**: In this problem, you're given a list of cities, and the goal is to find the shortest possible route that visits each city once and returns to the starting city. It's an NP-complete problem. If you could find the shortest route quickly, you could also solve other hard problems quickly.

NP-hard

- **NP-hard** problems are at least as hard as the hardest problems in NP, but they might **not be in NP** themselves. This means that, while NP-hard problems are very tough to solve, they might not have a solution that can be easily verified.
- In short, NP-hard problems are the most challenging, but they might not even have a "yes" or "no" answer to verify a solution against, like in NP problems.

Example:

- **Halting Problem**: Given a computer program and its input, determine whether the program will eventually stop or keep running forever. This is NP-hard, but there's no easy way to verify the solution because it's undecidable.

Assignment-Cum-Tutorial Questions

Part-A
Objective Questions (1M)

1. A Turing Machine consists of: [ ]

A) Finite tape
B) Infinite tape
C) Stack
D) Queue

2. Which component of a TM reads and writes symbols? [ ]

A) Stack
B) CPU
C) Read/Write Head
D) Cache

3. A Turing Machine is represented by: [ ]

A) 5-tuple
B) 6-tuple
C) 7-tuple
D) 8-tuple

4. In the tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$, Γ represents: [ ]

A) States
B) Input alphabet
C) Tape alphabet
D) Final state

5. An Instantaneous Description represents [ ]
A) Grammar derivation
B) Current configuration of TM
C) Parse tree
D) PDA stack

6. A language accepted by a TM is called [ ]
A) Regular language
B) Context-free language
C) Recursively Enumerable Language
D) Finite language

7. Which TM has multiple tapes? [ ]

A) Universal TM

B) Multi-tape TM

C) Restricted TM

D) Enumerator

8. Universal TM can simulate [ ]

A) DFA only

B) PDA only

C) Any Turing Machine

D) CFG

9. Church-Turing Thesis is [ ]

A) A theorem

B) A hypothesis

C) An algorithm

D) A grammar

10. The proof of Halting Problem uses [ ]

A) Pumping Lemma

B) Contradiction and self-reference

C) Dynamic Programming

D) Greedy Algorithm

11. Class P consists of problems solvable in [ ]

A) Polynomial time

B) Exponential time

C) Logarithmic time only

D) Constant time

12. If any NP-Complete problem is solved in polynomial time, then [ ]

A) P ≠ NP

B) P = NP

C) NP becomes undecidable

D) All languages become regular

Part-B
Short Answer Questions (2M)

1. Define a Turing Machine (TM).
2. What is the purpose of the blank symbol in a TM?
3. Define an Instantaneous Description (ID).
4. List different types of turing machines.
5. State Church Thesis.
6. Differentiate between Universal TM and Restricted TM.
7. Define the Halting Problem.
8. Define Post Correspondence Problem
9. Define class P.
10. What is an NP problem?
11. Define NP complete and NP Hard.
12. State the difference between PCP and MPCP.

Part C
Descriptive Questions (5M)

1. Construct Turing Machine for $L=\{ww^R \mid w \in (0, 1)\}$
2. Design a Turing Machine to accept the language $L = \{a^n b^n \mid n \geq 1\}$.

3. Design a Turing Machine to accept the language L = $\{a^n b^{2n} \mid n \geq 1\}$.
4. Design a Turing Machine to accept the language L = $\{a^n b^n c^n \mid n \geq 1\}$.
5. Design a TM to accept the language L = $\{0^n 1^n \mid n \geq 1\}$.
6. Design a TM to accept the language L = $\{0^{2n} 1^n \mid n \geq 1\}$.
7. Design Turing Machine for 1's complement of binary number.
8. Design Turing Machine for 2's complement of binary number.
9. Design Turing Machine for addition of unary numbers.
10. Design Turing Machine for subtraction of unary numbers.
11. Design a Turing Machine for palindrome strings over {a,b}.
12. Design a Turing Machine to convert lowercase to uppercase.
13. Explain different types of Turing Machines with suitable examples.
14. Write the difference between finite automata and Turing machine.
15. Illustrate the model of Turing Machine with a neat sketch.
16. State and explain Post's Correspondence Problem (PCP). Construct an example and show how it can be solved.
17. Find whether there is a solution for the following Post's Correspondence Problem instance. A={b^3 ,ab^2 } B={b^3 ,bab^3 }.
18. Find whether there is a solution for the following PCP instance. A={1,10111,10}, B={111,10,0}.
19. Check whether the following post correspondence problem has a solution or not.

I	List A	List B
1	111	11
2	1001	01
3	1110	1001

20. Describe Universal and Restricted Turing Machines.
21. Explain Church's Thesis in detail.
22. Explain the classes P and NP. Give suitable examples for each.
23. Differentiate NP-Hard and NP-Complete problems with examples.

Part D
Competitive Exam Questions

1. The transition function of a standard Turing Machine is $\delta: Q \times \Gamma \rightarrow$ [ ]
   A) Q
   B) $Q \times \Gamma Q$
   C) $Q \times \Gamma \times \{L,R\}$
   D) $Q \times \Sigma$

2. Which of the following is TRUE? [ ]

A) Every recursive language is recursively enumerable.

B) Every recursively enumerable language is recursive.

C) Every undecidable language is recursive.

D) Recursive languages are not recursively enumerable.

3. A deterministic Turing Machine has 6 states (including accept and reject states) and a tape alphabet of 4 symbols. The maximum number of transitions that can be defined is [ ]

A) 20

B) 24

C) 32

D) 36

4. If a TM has 5 states and tape alphabet of size 6 .How many entries are present in its transition table? [ ]

A) 11

B) 25

C) 30

D) 36

5. The Halting Problem is [ ]

A) Recursive

B**) Recursively enumerable but undecidable**

C) NP-Complete

D) Regular